

H524 Introduction to Biostatistics

Week 1 Lab: Introduction to R and Data Visualization

Fall 2025

Lab Duration: 110 minutes (8:00am – 9:50am)

Format: Online

Tools Required: R, RStudio, GitHub account, GitHub Copilot (optional)

1 Learning Objectives

By the end of this lab, you will be able to:

1. Set up R and RStudio for biostatistical analysis
2. Create and manipulate basic R data structures
3. Generate descriptive statistics for health data
4. Create publication-quality visualizations for different data types
5. [Use GitHub Copilot to enhance R coding productivity](#)
6. [Critically evaluate and verify AI-generated code](#)

2 Lab Setup

2.1 Required Software

1. **R:** Download from <https://cran.r-project.org/>
2. **RStudio:** Download from <https://posit.co/products/open-source/rstudio/>
3. **GitHub Account:** Sign up at <https://github.com/>
4. **GitHub Copilot:** Available free for students at <https://education.github.com/>

2.2 Verify Installation

Open RStudio and run the following code to verify your setup:

```
1 # Check R version
2 R.version.string
3
4 # Check working directory
5 getwd()
6
7 # Test basic functionality
8 x <- c(1, 2, 3, 4, 5)
9 mean(x)
```

3 Part 1: R Fundamentals (30 minutes)

3.1 Data Types and Structures

Exercise 1.1: Create variables of different data types relevant to health research.

```
1 # Numeric data
2 systolic_bp <- c(120, 135, 142, 118, 160, 125, 138)
3 diastolic_bp <- c(80, 85, 92, 75, 95, 82, 88)
4
5 # Character data
6 patient_id <- c("P001", "P002", "P003", "P004", "P005", "P006", "P007")
7 gender <- c("Male", "Female", "Female", "Male", "Male", "Female", "Male")
8
9 # Factor data (categorical)
10 treatment_group <- factor(c("Control", "Treatment", "Treatment",
11                             "Control", "Treatment", "Control", "Treatment")
12                             )
13
14 # Logical data
15 hypertensive <- systolic_bp >= 140 | diastolic_bp >= 90
16
17 # Display the data types
18 class(systolic_bp)
19 class(gender)
20 class(treatment_group)
21 class(hypertensive)
```

Exercise 1.2: Create a data frame combining health variables.

```
1 # Create a comprehensive health dataset
2 health_data <- data.frame(
3   patient_id = patient_id,
4   gender = gender,
5   systolic_bp = systolic_bp,
6   diastolic_bp = diastolic_bp,
7   treatment_group = treatment_group,
8   hypertensive = hypertensive
9 )
10
11 # View the data structure
12 str(health_data)
13 head(health_data)
14 summary(health_data)
```

4 Part 2: Descriptive Statistics (25 minutes)

Exercise 2.1: Calculate measures of central tendency and variability.

```
1 # Measures of central tendency
2 mean_systolic <- mean(health_data$systolic_bp)
3 median_systolic <- median(health_data$systolic_bp)
4
5 # Measures of variability
```

```

6 sd_systolic <- sd(health_data$systolic_bp)
7 var_systolic <- var(health_data$systolic_bp)
8 range_systolic <- range(health_data$systolic_bp)
9 iqr_systolic <- IQR(health_data$systolic_bp)
10
11 # Display results
12 cat("Mean systolic BP:", round(mean_systolic, 2), "mmHg\n")
13 cat("Median systolic BP:", median_systolic, "mmHg\n")
14 cat("Standard deviation:", round(sd_systolic, 2), "mmHg\n")
15 cat("Range:", range_systolic[1], "-", range_systolic[2], "mmHg\n")

```

Exercise 2.2: Generate descriptive statistics by group.

```

1 # Statistics by gender
2 aggregate(systolic_bp ~ gender, data = health_data, FUN = mean)
3 aggregate(systolic_bp ~ gender, data = health_data, FUN = sd)
4
5 # Statistics by treatment group
6 aggregate(systolic_bp ~ treatment_group, data = health_data, FUN = summary
7 )
8
9 # Frequency tables for categorical data
10 table(health_data$gender)
11 table(health_data$treatment_group)
12 prop.table(table(health_data$hypertensive))

```

5 Part 3: Data Visualization (35 minutes)

5.1 Visualizations for Continuous Data

Exercise 3.1: Create histograms and box plots.

```

1 # Histogram of systolic blood pressure
2 hist(health_data$systolic_bp,
3     main = "Distribution of Systolic Blood Pressure",
4     xlab = "Systolic BP (mmHg)",
5     ylab = "Frequency",
6     col = "lightblue",
7     border = "white",
8     breaks = 5)
9
10 # Add vertical line for mean
11 abline(v = mean_systolic, col = "red", lwd = 2, lty = 2)
12 legend("topright", "Mean", col = "red", lwd = 2, lty = 2)
13
14 # Box plot by gender
15 boxplot(systolic_bp ~ gender,
16     data = health_data,
17     main = "Systolic BP by Gender",
18     ylab = "Systolic BP (mmHg)",
19     col = c("pink", "lightblue"))

```

5.2 Visualizations for Categorical Data

Exercise 3.2: Create bar charts and pie charts.

```
1 # Bar chart for treatment groups
2 treatment_counts <- table(health_data$treatment_group)
3 barplot(treatment_counts,
4         main = "Distribution of Treatment Groups",
5         ylab = "Frequency",
6         col = c("#FF6B35", "#F7931E", "#FFAA1D"))
7
8 # Pie chart for gender distribution
9 gender_counts <- table(health_data$gender)
10 pie(gender_counts,
11     main = "Gender Distribution",
12     col = c("pink", "lightblue"))
```

5.3 Advanced Visualizations

Exercise 3.3: Create scatter plots and add trend lines.

```
1 # Scatter plot of systolic vs diastolic BP
2 plot(health_data$diastolic_bp, health_data$systolic_bp,
3      xlab = "Diastolic BP (mmHg)",
4      ylab = "Systolic BP (mmHg)",
5      main = "Systolic vs Diastolic Blood Pressure",
6      pch = 19,
7      col = as.numeric(health_data$gender))
8
9 # Add legend
10 legend("topleft", levels(health_data$gender),
11       col = 1:2, pch = 19)
12
13 # Add regression line
14 abline(lm(systolic_bp ~ diastolic_bp, data = health_data),
15       col = "red", lwd = 2)
```

6 Part 4: AI-Enhanced Coding with GitHub Copilot (20 minutes)

AI Integration Component

This section introduces you to using AI tools (GitHub Copilot) to enhance your R programming productivity while maintaining statistical rigor.

6.1 Setting Up GitHub Copilot in RStudio

Exercise 4.1: Configure GitHub Copilot (if available).

1. Install the GitHub Copilot extension in RStudio (if supported)
2. Alternatively, use GitHub Copilot in VS Code with R extension
3. Sign in with your GitHub account

6.2 AI-Assisted Code Generation

Exercise 4.2: Use AI to generate statistical code.

Prompt Example: Type a comment describing what you want to do:

```
1 # Create a function to calculate age-adjusted rates for three age groups
2 # Age groups: 0-30, 31-60, 61+
3 # Input: cases vector, population vector, age_groups vector
4 # Output: age-specific rates per 100,000
5
6 # Let GitHub Copilot suggest the function
```

Exercise 4.3: Verify and test AI-generated code.

```
1 # Test the AI-generated function with sample data
2 test_cases <- c(10, 25, 40)
3 test_population <- c(20000, 30000, 15000)
4 test_ages <- c("0-30", "31-60", "61+")
5
6 # Run the function and verify results manually
7 # Calculate: (cases/population) * 100000
8 manual_check <- (test_cases / test_population) * 100000
9 print(manual_check)
10
11 # Compare with AI function output
12 # [Test the AI-generated function here]
```

6.3 AI Best Practices for Biostatistics

Exercise 4.4: Critical evaluation checklist.

1. **Review AI suggestions:** Does the code make statistical sense?
2. **Test with known data:** Verify results with manual calculations
3. **Check assumptions:** Are statistical assumptions appropriate?
4. **Validate output:** Do results align with domain knowledge?

AI Verification Task: For each AI-generated code snippet:

- Document the prompt used
- Test with sample data
- Verify one calculation manually
- Note any corrections needed

7 Lab Assignment

7.1 Data Analysis Task

Use the FEV1 lung function data from the course materials:

```

1 # FEV1 data from Pagano & Gauvreau study
2 fev1_data <- data.frame(
3   subject_id = 1:13,
4   gender = c("F", "F", "F", "F", "F", "F", "M", "M", "M", "M", "M", "M", "M"),
5   fev1_liters = c(2.46, 2.65, 2.69, 2.76, 2.85, 3.15, 2.47, 2.77, 2.78,
6                  2.81, 3.09, 3.18, 3.46)
7 )

```

Assignment Tasks:

1. Calculate descriptive statistics (mean, median, SD, range) for FEV1 by gender
2. Create appropriate visualizations:
 - Histogram of all FEV1 values
 - Box plot comparing FEV1 by gender
 - Dot plot showing individual values
3. **AI Component:** Use AI to help generate code for:
 - A function to categorize FEV1 values as "Low" (<2.5), "Normal" (2.5-3.2), or "High" (>3.2)
 - Advanced visualization with better formatting
4. Document your AI usage:
 - What prompts did you use?
 - How did you verify the AI output?
 - What corrections were needed?

7.2 Deliverables

Submit the following files via Canvas:

1. week1_lab_[lastname].R - R script with all exercises
2. week1_lab_report_[lastname].pdf - Brief report (1-2 pages) including:
 - Summary statistics tables
 - Key visualizations
 - AI usage documentation
 - Brief interpretation of FEV1 findings

8 Additional Resources

- **R Documentation:** <https://www.rdocumentation.org/>
- **RStudio Cheatsheets:** <https://posit.co/resources/cheatsheets/>
- **GitHub Copilot Documentation:** <https://docs.github.com/en/copilot>
- **Biostatistics with R:** <https://cran.r-project.org/web/views/MedicalImaging.html>

9 Lab Reflection Questions

1. How did using AI tools change your approach to coding in R?
2. What were the main challenges in verifying AI-generated statistical code?
3. When would you prefer to write code manually vs. using AI assistance?
4. How can AI tools best support learning biostatistics concepts?

Remember: AI tools are powerful assistants, but you remain responsible for understanding the statistical methods and verifying all results. Always think critically about AI suggestions and test them thoroughly.